HANZO POST-QUANTUM CRYPTOGRAPHY

Migration Path for AI Infrastructure

Zach Kelling*Hanzo Industries Inc (Techstars '17)*research@hanzo.ai

April 2026

Abstract

We describe the post-quantum cryptographic migration of Hanzo infrastructure, replacing classical algorithms vulnerable to quantum attack with NIST-standardized post-quantum alternatives. The migration covers three domains: (i) ML-KEM (Module-Lattice-Based Key Encapsulation Mechanism, formerly Kyber) for key exchange in TLS connections between Hanzo services, client API connections, and KMS envelope encryption, (ii) ML-DSA (Module-Lattice-Based Digital Signature Algorithm, formerly Dilithium) for API request signing, IAM JWT tokens, and agent credential chains, and (iii) SLH-DSA (Stateless Hash-Based Digital Signature Algorithm, formerly SPHINCS+) as a conservative fallback for long-lived root certificates. We present the hybrid transition strategy where classical and post-quantum algorithms run in parallel during migration, the performance impact on Hanzo Gateway request latency (1.2ms increase for ML-KEM handshake, 0.8ms for ML-DSA verification), and the key size implications for KMS storage and IAM token payloads. Production deployment began in January 2026 with ML-KEM for inter-service TLS; full migration completes by Q4 2026.

1 Introduction

Cryptographically relevant quantum computers (CRQCs) threaten the security foundations of all modern infrastructure. Shor's algorithm [8] breaks RSA, ECDSA, and ECDH in polynomial time. Grover's algorithm [9] weakens symmetric ciphers and hash functions by a quadratic factor.

Hanzo infrastructure relies on cryptography at every layer:

- **TLS**: ECDHE key exchange (X25519) and ECDSA server certificates for all service-to-service and client-to-service connections.
- **IAM**: Ed25519 JWT signing for authentication tokens.
- **KMS**: AES-256-GCM envelope encryption with ECDH-derived key wrapping for secret storage.
- **Agent credentials**: Ed25519 certificate chains for delegated agent permissions.
- **ACI**: ECDSA (secp256k1) for blockchain transaction signing and compute attestations.

While large-scale CRQCs do not yet exist, the "harvest now, decrypt later" threat model requires action today: adversaries recording encrypted traffic can decrypt it once quantum computers become available. Data with long confidentiality requirements (medical records, financial data, government communications) processed through Hanzo AI must be protected against future quantum decryption.

1.1 NIST Standardization

NIST finalized three post-quantum standards in August 2024 [7]:

1. **ML-KEM** (FIPS 203): Lattice-based key encapsulation, replacing ECDH for key exchange.
2. **ML-DSA** (FIPS 204): Lattice-based digital signatures, replacing ECDSA/Ed25519 for signing.
3. **SLH-DSA** (FIPS 205): Hash-based digital signatures, providing a conservative alternative based on minimal assumptions.

2 Threat Model

Definition 1 (Quantum Threat Timeline)**.** *Let T_Q be the year a CRQC capable of breaking 256-bit elliptic curve cryptography becomes available. Let T_D be the required data confidentiality horizon. Data encrypted today is vulnerable if $T_D > T_Q$.*

Conservative estimates place T_Q between 2030 and 2040 [10]. For Hanzo infrastructure:

- **Healthcare AI data**: $T_D = 50+$ years (HIPAA requirements). Migration urgency: *critical*.
- **Financial AI data**: $T_D = 7$–25 years (regulatory retention). Migration urgency: *high*.
- **Session tokens**: $T_D < 24$ hours (short-lived). Migration urgency: *moderate* (but required for compliance).
- **Blockchain signatures**: $T_D = \infty$ (on-chain forever). Migration urgency: *critical*.

Theorem 1 (Migration Deadline)**.** *For data with confidentiality horizon T_D and migration timeline T_M (years to complete migration), the latest safe migration start date is $T_Q - T_M - T_D$. With $T_Q = 2033$ (median estimate), $T_M = 2$ years, and $T_D = 10$ years: migration must begin by 2021. We are already in the vulnerability window.*

3 ML-KEM for Key Exchange

ML-KEM (FIPS 203) replaces ECDH key exchange. It is based on the Module Learning with Errors (MLWE) problem.

Definition 2 (ML-KEM)**.** *ML-KEM consists of three algorithms:*

- `KeyGen`$() \to (ek, dk)$*: Generate encapsulation and decapsulation keys.*
- `Encaps`$(ek) \to (K, c)$*: Produce shared secret K and ciphertext c.*
- `Decaps`$(dk, c) \to K$*: Recover shared secret from ciphertext.*

We deploy ML-KEM-768 (NIST Security Level 3, equivalent to AES-192):

Parameter	**ML-KEM-768**	**X25519**	**Ratio**
Public key (bytes)	1,184	32	37×
Ciphertext (bytes)	1,088	32	34×
Shared secret (bytes)	32	32	1×
KeyGen (μs)	28	48	0.6×
Encaps (μs)	35	48	0.7×
Decaps (μs)	32	48	0.7×

Table 1: ML-KEM-768 vs X25519 parameter comparison. ML-KEM is computationally faster but has larger key/ciphertext sizes.

3.1 Hybrid Key Exchange

During the transition period, we use hybrid key exchange combining X25519 and ML-KEM-768:

$$K_{\text{hybrid}} = \texttt{HKDF-SHA256}(K_{\text{X25519}} \| K_{\text{ML-KEM}}), \tag{1}$$

ensuring security if either scheme is broken. This follows the IETF hybrid approach [11].

3.2 TLS Integration

Hanzo Gateway [1] implements hybrid TLS 1.3 with the `X25519MLKEM768` named group:

1. `ClientHello` includes both X25519 and ML-KEM-768 key shares (total 1,248 bytes vs. 32 bytes for X25519 alone).
2. `ServerHello` includes both encapsulated keys.
3. The handshake shared secret combines both derivations.
4. Application data is encrypted with AES-256-GCM (quantum-safe at 128-bit security via Grover's bound).

Remark 1. *The larger ClientHello (1,248 vs. 32 bytes) fits within a single TCP segment and does not require an additional round trip. Measured TLS handshake overhead: 1.2ms additional latency (from 3.1ms to 4.3ms for a cold connection).*

3.3 KMS Key Wrapping

Hanzo KMS [3] envelope encryption currently wraps data encryption keys (DEKs) using ECDH-derived keys. The post-quantum migration replaces the key wrapping:

1. Master KEK is derived via ML-KEM encapsulation.
2. DEK encryption uses AES-256-GCM with the ML-KEM-derived key.
3. Existing secrets are re-wrapped during the next rotation cycle (within 90 days per KMS rotation policy).

4 ML-DSA for Digital Signatures

ML-DSA (FIPS 204) replaces Ed25519 and ECDSA for digital signatures. It is based on the Module Learning with Errors and Module Short Integer Solution (MLWE/MSIS) problems.

Definition 3 (ML-DSA)**.** *ML-DSA consists of three algorithms:*

- $\texttt{KeyGen}() \to (vk, sk)$*: Generate verification and signing keys.*
- $\texttt{Sign}(sk, m) \to \sigma$*: Produce signature* σ *on message* m*.*
- $\texttt{Verify}(vk, m, \sigma) \to \{0, 1\}$*: Verify signature validity.*

We deploy ML-DSA-65 (NIST Security Level 3):

Parameter	ML-DSA-65	Ed25519	Ratio
Public key (bytes)	1,952	32	61×
Signature (bytes)	3,309	64	52×
Sign (μs)	120	22	5.5×
Verify (μs)	45	18	2.5×

Table 2: ML-DSA-65 vs Ed25519 parameter comparison.

4.1 IAM JWT Migration

Hanzo IAM [2] JWT tokens currently use Ed25519 signatures. The migration proceeds in three phases:

1. **Phase 1 (current)**: IAM signs with both Ed25519 and ML-DSA-65. Tokens carry dual signatures. Verifiers accept either.
2. **Phase 2 (Q3 2026)**: New tokens are signed with ML-DSA-65 only. Verifiers accept both algorithms (backward compatibility for tokens issued in Phase 1).
3. **Phase 3 (Q4 2026)**: Ed25519 verification is removed. All tokens are ML-DSA-65.

Remark 2. *The larger ML-DSA-65 signature (3,309 bytes vs. 64 bytes) increases JWT size from ∼500 bytes to ∼3,800 bytes. This remains well within HTTP header limits and does not impact API performance measurably.*

4.2 API Request Signing

External API requests to Hanzo Gateway use HMAC-SHA256 for authentication (quantum-safe at 128-bit security). Internal service-to-service requests use mTLS with ML-DSA-65 certificates.

4.3 Agent Credential Chains

Agent credentials [4] use certificate chains signed with Ed25519. Migration replaces the signature algorithm while preserving the chain structure:

$$\text{Root}_{ML\text{-}DSA} \rightarrow \text{Org}_{ML\text{-}DSA} \rightarrow \text{Agent}_{ML\text{-}DSA}. \tag{2}$$

During the hybrid period, cross-signed certificates bridge old and new root keys.

5 SLH-DSA for Root Certificates

SLH-DSA (FIPS 205) provides a conservative backup for long-lived root certificates. Unlike ML-DSA (lattice-based), SLH-DSA security relies only on hash function collision resistance—an assumption with decades of confidence.

Definition 4 (SLH-DSA Security Basis)**.** *SLH-DSA is secure if the underlying hash function (SHA-256 or SHAKE-256) is second-preimage resistant, a property believed to hold even against quantum adversaries (Grover reduces security by at most a quadratic factor).*

We use SLH-DSA-SHA2-192s (NIST Level 3, "small" variant) for root certificates:

Parameter	SLH-DSA-192s	ML-DSA-65
Public key (bytes)	48	1,952
Signature (bytes)	16,224	3,309
Sign (ms)	180	0.12
Verify (ms)	4.2	0.045

Table 3: SLH-DSA-192s vs ML-DSA-65. SLH-DSA has smaller keys but much larger signatures and slower operations.

Root certificates are verified rarely (once per TLS session establishment, then cached), so the 4.2ms verification overhead is acceptable.

6 ACI and Blockchain Migration

Blockchain signatures present unique challenges: transactions signed today are verified forever. A quantum adversary can forge historical transactions if the signing algorithm is broken.

6.1 Lux Network Coordination

The Lux blockchain [6] is independently migrating to post-quantum signatures. Hanzo ACI coordinates with Lux for:

- **Consensus signatures**: Validator attestations migrate from ECDSA to ML-DSA in a coordinated network upgrade.
- **Transaction signatures**: New transaction types support ML-DSA-65 signatures alongside legacy ECDSA (secp256k1).
- **Address derivation**: Post-quantum addresses are derived from ML-DSA-65 public keys, distinct from legacy ECDSA addresses.

6.2 Threshold Signing Migration

Hanzo Threshold Signing [5] migrates from FROST (Ed25519) and CGGMP21 (ECDSA) to post-quantum threshold schemes:

- **Lattice-based threshold signatures**: Research prototypes of threshold ML-DSA exist but are not yet production-ready. Expected standardization: 2027.
- **Interim approach**: Hash-based threshold signatures using SLH-DSA with Shamir secret sharing of the hash-based signing key. This is less efficient but provably secure.

7 Performance Impact

Operation	Classical	PQ Hybrid	Overhead
TLS handshake (cold)	3.1ms	4.3ms	+1.2ms
TLS handshake (resumed)	0.8ms	0.8ms	+0.0ms
JWT signing	0.022ms	0.142ms	+0.120ms
JWT verification	0.018ms	0.063ms	+0.045ms
API request (end-to-end)	12ms	13.2ms	+1.2ms
KMS secret fetch	5ms	5.2ms	+0.2ms

Table 4: Performance impact of post-quantum migration. Overhead is within acceptable bounds for all operations.

The total API request overhead of 1.2ms (from 12ms to 13.2ms) represents a 10% increase, dominated by the TLS handshake on cold connections. Persistent connections (used for all service-to-service communication) see negligible overhead.

8 Key Size and Storage Impact

Post-quantum key sizes significantly exceed classical equivalents. Impact on Hanzo systems:

- **KMS**: Each ML-KEM-768 public key stored alongside the classical X25519 key adds 1,184 bytes per secret. For 12,000 secrets: 14MB additional storage (negligible).
- **IAM JWKS endpoint**: ML-DSA-65 public keys in the JWKS response increase payload from 200 bytes to 2.2KB. Cached by verifiers.
- **JWT tokens**: Dual-signed tokens (Ed25519 + ML-DSA-65) grow from ∼500 bytes to ∼4.3KB. Within HTTP header limits.
- **Agent credential chains**: 3-level ML-DSA-65 chains total ∼16KB (3 signatures + 3 public keys). Cached per agent session.

9 Related Work

Lux post-quantum [6] implements post-quantum cryptography at the consensus and transaction layer. Hanzo coordinates its infrastructure migration with Lux to ensure compatibility.

Google Chrome [12] deployed X25519MLKEM768 in TLS 1.3 starting in Chrome 124 (2024). Our TLS configuration is compatible with Chrome's implementation.

Signal [13] adopted the PQXDH protocol combining X25519 and ML-KEM-768 for post-quantum-secure key exchange. Our approach follows the same hybrid pattern.

NIST PQC [7] standardized ML-KEM, ML-DSA, and SLH-DSA in August 2024. We adopt all three standards at their Level 3 parameter sets.

10 Conclusion

Hanzo post-quantum migration replaces vulnerable classical cryptography across all infrastructure: ML-KEM-768 for key exchange (TLS, KMS), ML-DSA-65 for digital signatures (IAM, agent credentials, API signing), and SLH-DSA-192s for root certificates. The hybrid transition strategy ensures security if either classical or post-quantum schemes are broken. Performance overhead is bounded at 1.2ms for cold TLS handshakes and 0.045ms for signature verification. Migration began in January 2026 with inter-service TLS and completes by Q4 2026 with full ML-DSA JWT signing.

References

[1] Hanzo AI Research. Hanzo Gateway: API gateway with KrakenD backend. *Hanzo AI Research*, 2024.

[2] Hanzo AI Research. Hanzo IAM: Identity and access management for AI platforms. *Hanzo AI Research*, 2024.

[3] Hanzo AI Research. Hanzo KMS: Secrets management with per-environment rotation and audit trail. *Hanzo AI Research*, September 2021.

[4] Hanzo AI Research. Hanzo Agent SDK: Framework for building AI agents. *Hanzo AI Research*, 2025.

[5] Hanzo AI Research. Hanzo Threshold Signing: MPC key management for AI infrastructure. *Hanzo AI Research*, April 2026.

[6] Lux Partners. Lux: A scalable multi-consensus blockchain with post-quantum cryptography. *Lux Network Technical Report*, 2024.

[7] NIST. Post-quantum cryptography standardization. *NIST FIPS 203, 204, 205*, August 2024.

[8] P. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proc. FOCS*, 1994.

[9] L. Grover. A fast quantum mechanical algorithm for database search. In *Proc. STOC*, 1996.

[10] M. Mosca. Cybersecurity in an era with quantum computers: Will we be ready? *IEEE Security & Privacy*, 16(5):38–41, 2018.

[11] D. Stebila, S. Fluhrer, and S. Gueron. Hybrid key exchange in TLS 1.3. *IETF Internet-Draft*, 2024.

[12] Google Chrome Team. Post-quantum cryptography in Chrome. *Chromium Blog*, 2024.

[13] Signal Foundation. PQXDH: Post-quantum extended Diffie-Hellman. *Signal Technical Documentation*, 2023.